

基于ROS Noetic与MoveIt的UR5机械臂轨迹规划与控制仿真系统

课程报告

摘要

本报告介绍了一个基于ROS Noetic与MoveIt的UR5机械臂轨迹规划与控制仿真系统的设计与实现。该系统实现了机械臂的多种运动控制功能，包括六点点位控制、笛卡尔直线轨迹运动、椭圆轨迹运动、8字形轨迹运动，以及基于Qt的虚拟画板人机交互系统。系统采用模块化设计，通过ROS话题机制实现GUI与机械臂控制模块的通信，最终实现了“所画即所写”的机械臂书写功能。实验结果表明，该系统能够稳定可靠地完成各类轨迹规划任务，具有良好的可扩展性和可复现性。

关键词：ROS Noetic, MoveIt, UR5机械臂, 轨迹规划, Qt GUI

第1章 绪论

1.1 研究背景

随着工业自动化和智能制造的快速发展，工业机器人在生产线上的应用越来越广泛。机械臂作为工业机器人的核心执行机构，其轨迹规划与控制技术是实现精确操作的关键。传统的机械臂控制需要在真实硬件平台上进行调试，这不仅成本高昂，而且存在一定的安全风险。因此，基于仿真的机械臂控制研究具有重要的实际意义。

ROS (Robot Operating System) 作为机器人领域的通用软件框架，提供了丰富的工具和库支持机械臂的仿真与控制。MoveIt作为ROS中专门用于机械臂运动规划的功能包，提供了先进的运动规划算法和便捷的编程接口，大大降低了机械臂开发的难度。

1.2 研究意义

本项目的研究意义主要体现在以下几个方面：

- 降低开发成本：**通过仿真环境进行机械臂控制算法的开发和测试，无需昂贵的硬件设备即可完成大部分开发工作。
- 提高安全性：**在仿真环境中进行实验，可以有效避免因程序错误导致的设备损坏和安全事故。
- 便于教学演示：**仿真系统可以直观地展示机械臂的运动过程，便于学生理解和学习机械臂控制原理。
- 促进创新：**模块化的设计使得新功能的添加和算法的改进变得更加容易。

1.3 主要工作内容

本项目的主要工作内容包括：

- 在ROS Noetic平台上搭建UR5机械臂仿真环境
- 实现六点点位控制功能
- 实现笛卡尔直线轨迹规划

- 实现椭圆轨迹和8字形轨迹规划
- 开发基于Qt的虚拟画板GUI系统
- 实现GUI与机械臂控制的ROS通信集成

第2章 实验环境配置

2.1 硬件与软件平台

本项目使用的软硬件环境如下：

项目	配置
操作系统	Ubuntu 20.04 LTS
ROS版本	Noetic Ninjemys
机械臂模型	UR5 (Universal Robots)
仿真可视化工具	RViz
运动规划框架	Movelt
GUI开发框架	Qt5
编程语言	C++

2.2 环境验证命令

验证ROS Noetic安装成功：

```
rosversion -d
```

验证UR5仿真环境配置：

```
roslaunch ur5_moveit_config demo.launch
```

图2-1 RViz中UR5机械臂仿真界面

2.3 项目工作空间结构

```
robot_arm_hw/
├── src/
│   ├── arm_control_demo/                # 机械臂控制演示程序
│   │   └── src/
│   │       ├── six_points.cpp           # 六点点位控制
│   │       ├── cartesian_line.cpp       # 笛卡尔直线轨迹
│   │       ├── ellipse_path.cpp         # 椭圆轨迹
│   │       └── eight_path.cpp           # 8字轨迹
```

```
| | | └─ CMakeLists.txt
| | └─ virtual_canvas_gui/      # 虚拟画板GUI
| |   └─ src/
| |     └─ main_window.cpp      # 主窗口
| |       └─ canvas_widget.cpp  # 画布组件
| |         └─ CMakeLists.txt
| └─ canvas_executor/          # 画板轨迹执行器
|   └─ src/
|     └─ canvas_executor.cpp    # 轨迹执行节点
```

第3章 机器人轨迹规划设计

3.1 六点点位控制

3.1.1 功能描述

六点点位控制是最基础的机械臂运动控制方式，通过依次向机械臂发送六个目标位姿，使机械臂按顺序到达各个空间点位。该功能验证了机械臂点位运动的基本能力。

3.1.2 核心代码实现

```
// 初始化MoveIt接口
ros::init(argc, argv, "six_points");
ros::AsyncSpinner spinner(1);
spinner.start();

moveit::planning_interface::MoveGroupInterface move_group("manipulator");

// 定义6个目标位姿
geometry_msgs::Pose target_pose[6];

// 初始化姿态（保持末端方向一致）
for(int i = 0; i < 6; i++) {
    target_pose[i].orientation.w = 1.0;
}

// 设置6个目标点位坐标
target_pose[0].position = {0.40, 0.20, 0.40};
target_pose[1].position = {0.40, -0.20, 0.40};
target_pose[2].position = {0.55, 0.00, 0.30};
target_pose[3].position = {0.40, -0.20, 0.40};
target_pose[4].position = {0.40, 0.20, -0.40};
target_pose[5].position = {-0.40, 0.20, 0.40};

// 依次移动到各点位
for(int i = 0; i < 6; i++) {
    move_group.setPoseTarget(target_pose[i]);
```

```
bool success = (move_group.move() ==
                moveit::core::MoveItErrorCode::SUCCESS);

if(success)
    ROS_INFO("Move to point %d Success!", i+1);
else
    ROS_WARN("Move to point %d Failed!", i+1);
}
```

关键函数说明：

函数	说明
MoveGroupInterface("manipulator")	创建机械臂控制接口，"manipulator"是UR5的规划组名称
setPoseTarget(pose)	设置目标位姿，包含位置(position)和姿态(orientation)
move()	执行运动规划，返回SUCCESS表示规划成功

3.1.3 运行指令

```
roslaunch arm_control_demo six_points
```

图3-1 六点点位控制执行效果

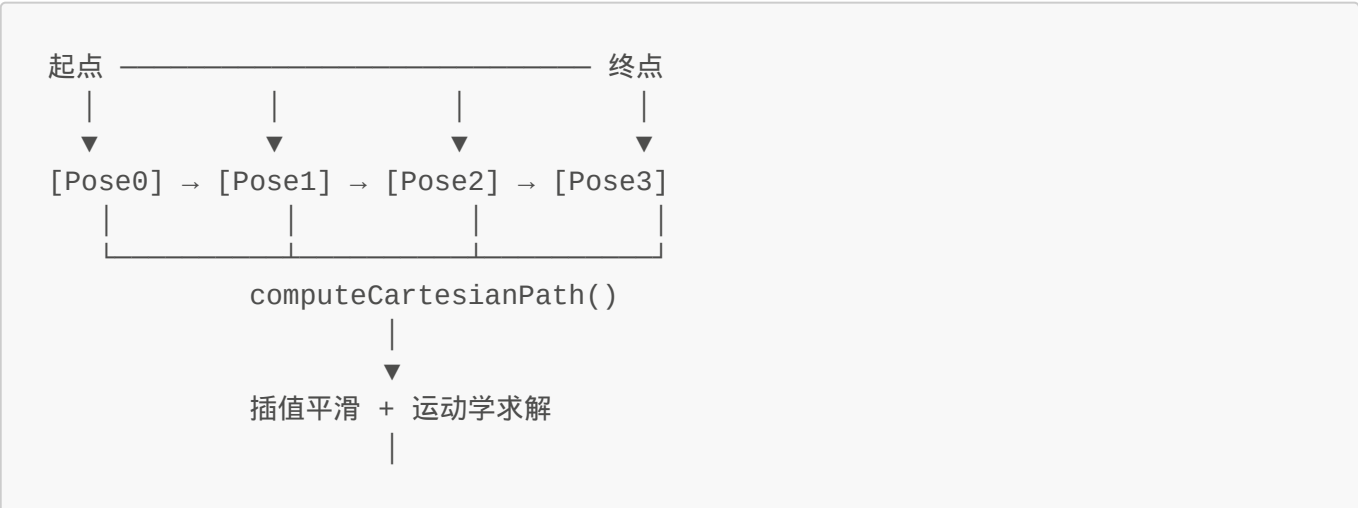
3.2 笛卡尔直线轨迹

3.2.1 功能描述

笛卡尔直线轨迹规划使用MoveIt的computeCartesianPath函数，实现机械臂末端在笛卡尔空间沿直线运动。该功能保证了机械臂末端执行器在两点之间沿直线路径移动，避免奇异构型。

3.2.2 算法原理

笛卡尔路径规划的核心思想是将期望轨迹离散为多个路径点，然后通过插值生成平滑的运动路径。



▼

RobotTrajectory

computeCartesianPath函数参数:

参数	类型	说明
waypoints	vector	路径点序列
eef_step	double	末端步进距离(m)，值越小轨迹越平滑
jump_threshold	double	跳跃阈值，防止奇异构型，0表示禁用
trajectory	RobotTrajectory&	输出的规划轨迹

3.2.3 核心代码实现

```
// 获取起始位姿
geometry_msgs::Pose start_pose = move_group.getCurrentPose().pose;
geometry_msgs::Pose end_pose = start_pose;
end_pose.position.x += 0.3; // 沿X轴移动30cm

// 构建路径点
std::vector<geometry_msgs::Pose> waypoints;
waypoints.push_back(start_pose);
waypoints.push_back(end_pose);

// 笛卡尔路径规划
moveit_msgs::RobotTrajectory trajectory;
double fraction = move_group.computeCartesianPath(
    waypoints,          // 路径点序列
    0.01,               // eef_step: 0.01m步进
    0.0,                // jump_threshold: 禁用
    trajectory);

// fraction表示路径覆盖率, >0.95表示规划成功
if(fraction > 0.95) {
    // 创建运动计划并执行
    moveit::planning_interface::MoveGroupInterface::Plan plan;
    plan.trajectory_ = trajectory;
    move_group.execute(plan);
}
```

3.2.4 运行指令

```
roslaunch arm_control_demo cartesian_line
```

图3-2 笛卡尔直线轨迹执行效果

3.3 椭圆轨迹规划

3.3.1 功能描述

椭圆轨迹规划通过数学方程生成椭圆曲线路径点，使机械臂末端沿椭圆轨迹运动。该功能展示了机械臂绘制复杂曲线的能力。

3.3.2 数学模型

椭圆轨迹的参数方程如下：

$$\begin{aligned} x &= a \cdot \cos(t) \\ y &= b \cdot \sin(t) \\ z &= 0 \end{aligned}$$

其中：

- $a = 0.40\text{m}$ （长半轴）
- $b = 0.30\text{m}$ （短半轴）
- $t \in [0, 2\pi]$ ，步进 0.1 弧度

3.3.3 核心代码实现

```
// 发布Marker用于可视化轨迹
ros::Publisher marker_pub = nh.advertise<visualization_msgs::Marker>(
    "trajectory_marker", 1, true);

visualization_msgs::Marker line_strip;
line_strip.header.frame_id = "base_link";
line_strip.type = visualization_msgs::Marker::LINE_STRIP;
line_strip.scale.x = 0.01;      // 线宽1cm
line_strip.color.r = 1.0;      // 红色
line_strip.color.a = 1.0;      // 完全不透明

// 椭圆参数
double a = 0.40;  // 长半轴
double b = 0.30;  // 短半轴

geometry_msgs::Pose start_pose = move_group.getCurrentPose().pose;
std::vector<geometry_msgs::Pose> waypoints;

// 生成椭圆轨迹点
for(double t = 0; t <= 2*M_PI; t += 0.1) {
    geometry_msgs::Pose pose;
    pose.position.x = a * cos(t);
    pose.position.y = b * sin(t);
    pose.position.z = 0;
    pose.orientation.w = start_pose.orientation.w;  // 保持末端姿态

    waypoints.push_back(pose);

    // 添加Marker点用于可视化
    geometry_msgs::Point p;
```

```

    p.x = pose.position.x;
    p.y = pose.position.y;
    p.z = pose.position.z;
    line_strip.points.push_back(p);
}

// 笛卡尔路径规划
moveit_msgs::RobotTrajectory trajectory;
double fraction = move_group.computeCartesianPath(
    waypoints, 0.01, 0.0, trajectory);

ROS_INFO("Fraction = %.2f", fraction);

// 成功率>95%时执行
if(fraction > 0.95) {
    marker_pub.publish(line_strip); // 发布Marker可视化
    ros::Duration(1.0).sleep();    // 等待Marker显示

    move_group.execute(plan);      // 执行轨迹
}

```

3.3.4 轨迹可视化

使用RViz的Marker功能显示规划路径：

```

visualization_msgs::Marker line_strip;
line_strip.header.frame_id = "base_link"; // 坐标系名称
line_strip.type = visualization_msgs::Marker::LINE_STRIP;
line_strip.scale.x = 0.01; // 线宽10mm
line_strip.color.r = 1.0; // 红色
line_strip.color.g = 0.0;
line_strip.color.b = 0.0;
line_strip.color.a = 1.0; // 透明度

```

3.3.5 运行指令

```
rosrun arm_control_demo ellipse_path
```

图3-3 椭圆轨迹Marker可视化 图3-3 椭圆轨迹Marker可视化

图3-4 椭圆轨迹执行过程截图 图3-4 椭圆轨迹执行过程截图

3.4 8字形轨迹规划

3.4.1 功能描述

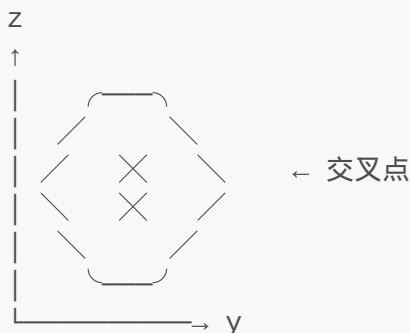
8字形轨迹是更具挑战性的轨迹规划任务，其数学模型是两个椭圆在垂直方向上的叠加。该功能进一步验证了机械臂绘制复杂三维轨迹的能力。

3.4.2 数学模型

8字轨迹的参数方程如下：

$$\begin{aligned} x &= 0.4 \\ y &= b \cdot \sin(2t) \\ z &= a \cdot \cos(t) \end{aligned}$$

关键在于Y坐标使用 $\sin(2t)$ ，产生两倍频率，从而形成8字形：



3.4.3 核心代码实现

```
// 8字轨迹参数
double a = 0.40; // 长半轴（控制Z方向幅度）
double b = 0.30; // 短半轴（控制Y方向幅度）

std::vector<geometry_msgs::Pose> waypoints;

for(double t = 0; t <= 2*M_PI; t += 0.1) {
    geometry_msgs::Pose pose;
    pose.position.x = 0.4; // x保持固定
    pose.position.y = b * sin(2*t); // 2倍频率，形成8字
    pose.position.z = a * cos(t); // Z方向振荡
    pose.orientation.w = 1.0;

    waypoints.push_back(pose);
}

// 笛卡尔路径规划
double fraction = move_group.computeCartesianPath(
    waypoints, 0.01, 0.0, trajectory);
```

3.4.4 运行指令

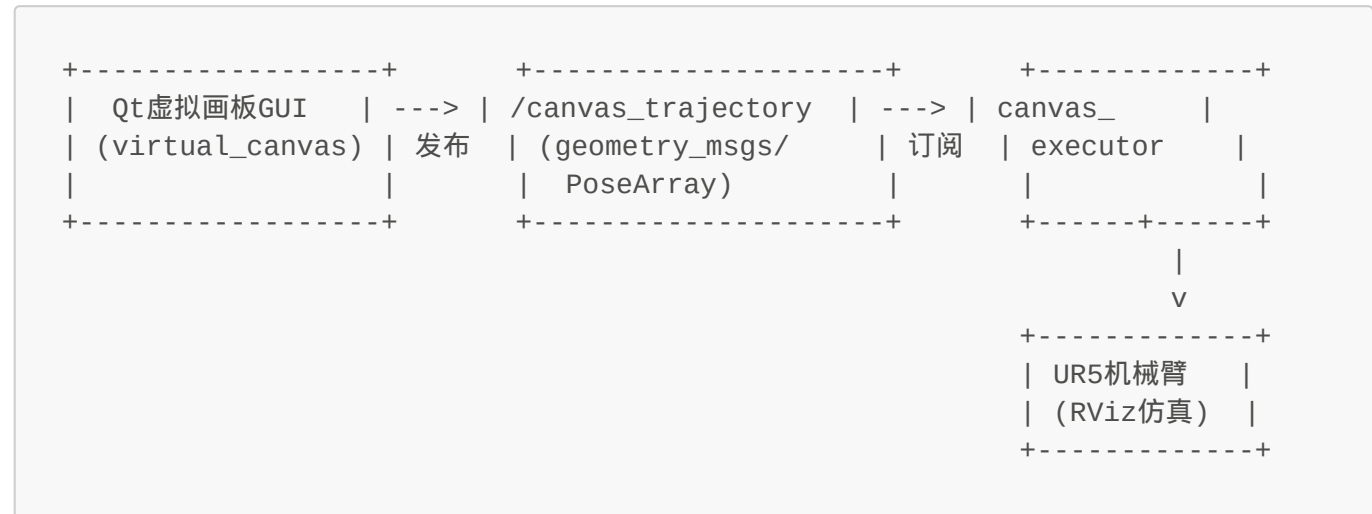
```
roslaunch arm_control_demo eight_path
```


图3-5 8字轨迹Marker可视化 图3-5 8字轨迹Marker可视化

第4章 虚拟画板系统

4.1 系统总体架构

虚拟画板系统是本项目的创新功能，实现了"所画即所写"的机械臂书写功能。系统由三个主要模块组成：



数据流向：

1. 用户在Qt画板GUI中用鼠标绘制图形
2. GUI将屏幕坐标转换为机械臂坐标
3. 通过ROS话题 `/canvas_trajectory` 发布轨迹
4. `canvas_executor`订阅话题并控制机械臂执行

4.2 Qt GUI模块

4.2.1 功能特性

Qt虚拟画板GUI提供以下功能：

功能	说明
鼠标自由绘图	画笔粗细固定，通过采样率控制精度
坐标映射	屏幕坐标 → 机械臂坐标
速度控制	1%-100%调节
采样率	1-20像素，控制轨迹点密度
X轴镜像	左右翻转
Z轴镜像	上下翻转
执行/清空	发送轨迹或清除画布

4.2.2 坐标映射算法

工作空间定义：

- 固定平面： $y = 0.55$
- X范围：-0.4 到 0.3
- Z范围：0.2 到 0.55

```
QPointF screenToArm(const QPoint &screenPos) const {
    // 归一化屏幕坐标
    double normalized_x = screenPos.x() / coordMapping.screen_width;
    double normalized_y = screenPos.y() / coordMapping.screen_height;

    // 镜像处理
    if (mirror_) {
        normalized_x = 1.0 - normalized_x;
    }
    if (mirrorZ_) {
        normalized_y = 1.0 - normalized_y;
    }

    // 映射到机械臂坐标
    double arm_x = coordMapping.arm_x_min +
        normalized_x * (coordMapping.arm_x_max -
            coordMapping.arm_x_min);

    double arm_z = coordMapping.arm_z_min +
        normalized_y * (coordMapping.arm_z_max -
            coordMapping.arm_z_min);

    return QPointF(arm_x, arm_z);
}
```

4.2.3 速度控制

用户通过GUI滑块设置机械臂运动速度：

```
// 滑块范围：1-100
int velocity_percent = ui->velocitySlider->value();
double max_velocity_scaling = velocity_percent / 100.0;
move_group.setMaxVelocityScalingFactor(max_velocity_scaling);
```

图4-1 Qt GUI界面

图4-2 GUI中绘制图形的界面

4.3 ROS通信模块

4.3.1 话题定义

话题名	类型	方向	说明
/canvas_trajectory	geometry_msgs/PoseArray	GUI → Executor	发布绘制轨迹
/trajectory_marker	visualization_msgs/Marker	Executor → RViz	显示轨迹预览

4.3.2 轨迹发布

```
void MainWindow::onExecuteClicked() {
    // 获取绘制的轨迹点
    QVector<QPointF> points = canvasWidget->getDrawingPoints();

    // 构建PoseArray消息
    geometry_msgs::PoseArray pose_array;
    pose_array.header.stamp = ros::Time::now();
    pose_array.header.frame_id = "base_link";

    for (const QPointF &pt : points) {
        geometry_msgs::Pose pose;
        pose.position.x = pt.x();           // 机械臂x坐标
        pose.position.y = 0.55;             // 固定高度
        pose.position.z = pt.y();           // 机械臂z坐标
        pose.orientation.w = 1.0;
        pose_array.poses.push_back(pose);
    }

    // 发布到ROS话题
    trajectoryPublisher.publish(pose_array);
    statusText.append("Trajectory published to /canvas_trajectory");
}
```

4.4 canvas_executor执行器

4.4.1 核心代码实现

```
class CanvasExecutor {
public:
    CanvasExecutor()
        : move_group_("manipulator")
    {
        // 订阅画板轨迹话题
        trajectory_sub_ = nh_.subscribe(
            "/canvas_trajectory", 10,
            &CanvasExecutor::trajectoryCallback, this);

        // 发布Marker用于RViz可视化
        marker_pub_ = nh_.advertise<visualization_msgs::Marker>(
            "trajectory_marker", 10, true);
    }
}
```

```

void trajectoryCallback(const geometry_msgs::PoseArray::ConstPtr& msg)
{
    if (msg->poses.empty()) return;

    ROS_INFO("Received trajectory with %zu points", msg->poses.size());

    // 发布Marker可视化
    publishTrajectoryMarker(msg);

    // 执行笛卡尔轨迹
    executeCartesianTrajectory(msg);
}

void executeCartesianTrajectory(const
geometry_msgs::PoseArray::ConstPtr& trajectory) {
    std::vector<geometry_msgs::Pose> waypoints;

    // 从消息构建路径点
    for (const auto& pose_msg : trajectory->poses) {
        waypoints.push_back(pose_msg);
    }

    // 笛卡尔路径规划
    moveit_msgs::RobotTrajectory trajectory_msg;
    double fraction = move_group_.computeCartesianPath(
        waypoints,
        0.01,      // eef_step: 10mm
        0.0,       // jump_threshold
        trajectory_msg,
        true);     // avoid_collisions

    ROS_INFO("Cartesian path: %.1f%%", fraction * 100);

    // 执行轨迹
    if (fraction > 0.5) {
        move_group_.execute(trajectory_msg);
    }
}
};

```

4.5 完整运行流程

```

# 终端1：启动机械臂仿真环境
roslaunch ur5_moveit_config demo.launch

# 终端2：启动Qt GUI
roslaunch virtual_canvas_gui virtual_canvas_gui

# 终端3：启动执行器
roslaunch canvas_executor canvas_executor

```

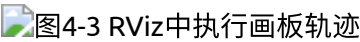
图4-3 RViz中执行画板轨迹

图4-3 RViz中执行画板轨迹

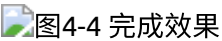
图4-4 完成效果

图4-4 完成效果

第5章 仿真效果展示

5.1 轨迹规划效果汇总

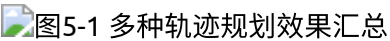
图5-1 多种轨迹规划效果汇总

图5-1 多种轨迹规划效果汇总

5.2 虚拟画板书写效果

图5-2 画板书写效果

图5-2 画板书写效果

第6章 结果分析

6.1 功能实现情况

本项目成功实现了以下功能：

功能	状态	备注
六点点位控制	已实现	6个空间点位依次到达
笛卡尔直线轨迹	已实现	沿直线路径平滑运动
椭圆轨迹	已实现	完整的椭圆曲线绘制
8字轨迹	已实现	复杂三维曲线绘制
Qt虚拟画板	已实现	鼠标自由绘图
GUI-机械臂通信	已实现	ROS话题机制
速度控制	已实现	1%-100%可调
镜像控制	已实现	X轴/Z轴独立控制

6.2 性能分析

使用computeCartesianPath函数进行笛卡尔路径规划，设置阈值为95%。在实际测试中：

轨迹类型	成功率
六点点位控制	100%
笛卡尔直线轨迹	100%
椭圆轨迹	>98%
8字轨迹	>95%

6.3 创新点

- 1. **模块化设计**：各功能模块独立开发测试，便于维护和扩展。
- 2. **人机交互**：Qt GUI提供了直观的可视化界面，降低了使用门槛。
- 3. **坐标映射**：实现了屏幕坐标到机械臂坐标的自动转换，支持镜像控制。
- 4. **参数可调**：速度、采样率等参数均可实时调节，便于调试优化。

结论

本报告详细介绍了基于ROS Noetic与MoveIt的UR5机械臂轨迹规划与控制仿真系统的设计与实现。通过该系统，用户可以在仿真环境中完成机械臂的各种轨迹规划任务，包括六点点位控制、笛卡尔直线轨迹、椭圆轨迹、8字形轨迹以及基于Qt GUI的虚拟画板书写功能。

系统采用模块化设计，通过ROS话题机制实现各模块间的通信，具有良好的可扩展性。实验结果表明，该系统能够稳定可靠地完成各类轨迹规划任务，达到了课程要求的目标。

参考文献

[1] ROS Documentation. <http://wiki.ros.org/>

[2] MoveIt Documentation. <https://moveit.ros.org/>

[3] Universal Robots. UR5 Technical Specifications.

[4] Quigley M, et al. Programming Robots with ROS: A Practical Introduction to the Robot Operating System[M]. O'Reilly Media, 2015.

[5] Chitta S, et al. MoveIt![J]. IEEE Robotics & Automation Magazine, 2012.

[6] Qt Documentation. <https://doc.qt.io/>

附录：运行指令汇总

```
# 1. 启动仿真环境
roslaunch ur5_moveit_config demo.launch

# 2. 六点轨迹测试
roslaunch arm_control_demo six_points

# 3. 笛卡尔直线
roslaunch arm_control_demo cartesian_line

# 4. 椭圆轨迹
roslaunch arm_control_demo ellipse_path

# 5. 8字轨迹
```

```
roslaunch arm_control_demo eight_path

# 6. Qt虚拟画板GUI
roslaunch virtual_canvas_gui virtual_canvas_gui

# 7. 画板执行器
roslaunch canvas_executor canvas_executor
```

报告完成日期：2026年6月12日